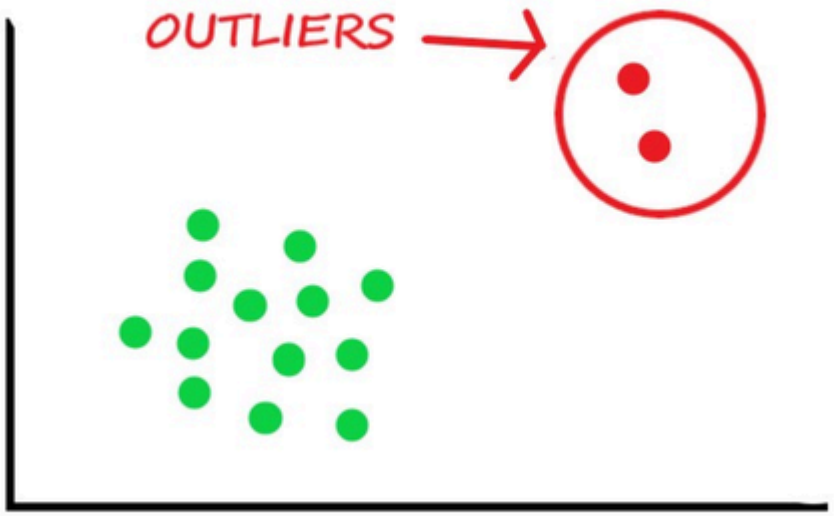
Introduction to Outlier

An outlier is an observation that is unlike the other observations. It is rare, or distinct, or does not fit in some way. It is also called anomalies.



Outliers are extreme values or unusual data points in a dataset that differ significantly from other observations. They are crucial to understand because they can affect model accuracy and lead to misleading insights if not properly addressed.

Types of Outliers

- **Univariate Outliers: These are unusual values in a single feature. For example, in a dataset of people's ages, an age of 120 would be an outlier if most values are between 0 and 100.**
- **Multivariate Outliers: These are unusual combinations of values across multiple features. For example, in a dataset of students' heights and weights, a student who is 6 feet**

tall and weighs 30 kg might be considered an outlier based on the combined information.

Why Outliers Matter

Outliers are important because they can:

- **Bias Analysis and Model Training: Many machine learning algorithms, such as linear regression and k-means clustering, are sensitive to outliers and might produce biased predictions due to these extreme values.**
- **Influence Statistical Measures: Outliers can skew mean and standard deviation, causing inaccurate representation of the data distribution.**
- **Reveal Anomalies or Errors: Outliers could indicate data entry errors, measurement inaccuracies, or significant insights such as fraudulent transactions or unusual customer behavior.**

Causes of Outliers

- **Data Entry Errors: Manual or automated errors during data collection, like entering an extra zero, can create outliers.**
- **Measurement Errors: Issues with data recording instruments, like faulty sensors, can lead to unusual values.**
- **Natural Variation: Some outliers are a natural part of the data, especially in fields with high variability (e.g., stock market data).**
- **Novel Insights or Rare Events: Outliers can represent valuable insights, like detecting fraudulent transactions or identifying specific population segments.**

What is the impact of Outliers on a dataset?

Outliers can drastically change the results of the data analysis and statistical modeling. There are numerous unfavourable impacts of outliers in the data set:

- It increases the error variance and reduces the power of statistical tests

- If the outliers are non-randomly distributed, they can decrease normality

- They can bias or influence estimates that may be of substantive interest

- They can also impact the basic assumption of Regression, ANOVA and other statistical model assumptions.

Without Outlier	With Outlier
4, 4, 5, 5, 5, 5, 6, 6, 6, 7, 7	4, 4, 5, 5, 5, 5, 6, 6, 6, 7, 7,300
Mean = 5.45	Mean = 30.00
Median = 5.00	Median = 5.50
Mode = 5.00	Mode = 5.00
Standard Deviation = 1.04	Standard Deviation = 85.03

Detecting Outliers

There are several methods to detect outliers, both visually and statistically:

Visual Methods

- **Box Plots: Outliers appear as points outside the "whiskers" (the lower and upper limits) of the box plot.**

- **Scatter Plots: Unusual values or points that fall far from other clusters of data can visually stand out.**
- **Histogram: Extreme bins in histograms, far away from the main data distribution, can indicate outliers.**

Statistical Methods

- **Z-Score: The Z-score measures the distance of a data point from the mean in terms of standard deviation. A Z-score above 3 or below -3 often indicates an outlier.**
  $$\text{Z-score} = \frac{(X - \text{mean})}{\text{standard deviation}}$$
- **Interquartile Range (IQR): The IQR method looks at the range between the 25th (Q1) and 75th (Q3) percentiles. Outliers are often defined as points outside the range:**
  $$\text{Outliers} = X < Q1 - 1.5 \times IQR \text{ or } X > Q3 + 1.5 \times IQR$$
- **Isolation Forest: A machine learning method specifically for identifying outliers by isolating points that are distinct from others.**
- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise): A clustering algorithm that can label low-density points (outliers) as noise, separate from clusters.**

Handling Outliers

Depending on the nature of the dataset and the goals of the analysis, outliers can be handled in several ways:

- **Remove: If the outliers are caused by errors or irrelevant noise, they may be removed. However, this should be done with caution, as valuable information might be lost.**

- **Cap or Floor Values: Extreme values can be set to a predefined cap or floor to reduce their impact. For example, setting all values above the 99th percentile to the 99th percentile value.**
- **Transform the Data: Log transformations, square root transformations, or normalization can help to reduce the influence of outliers, especially if the data is skewed.**
- **Use Robust Algorithms: Some algorithms are less sensitive to outliers, such as decision trees and Random Forests, which may handle outliers better than linear models.**
- **Imputation: In some cases, outliers can be replaced with more representative values, like the median or mean of the dataset, although this may dilute some of the data's uniqueness.**

Handling Outliers in Different Machine Learning Models

- **Regression Models: Outliers can disproportionately influence regression models by pulling the best-fit line toward the extreme values, which can lead to poor generalization. Removing or transforming outliers is often recommended.**
- **Classification Models: Outliers can lead to misclassification, especially in k-Nearest Neighbors (k-NN) or Logistic Regression. Handling outliers can improve classification accuracy.**
- **Clustering Models: Outliers can distort clusters in models like k-means, which rely on centroid calculation. Density-based clustering methods, like DBSCAN, are often better at dealing with outliers in clustering tasks.**

Example of Outlier Detection and Handling

Suppose you have a dataset of daily sales amounts. Most values are in the range of $50–$500, but a few entries are $10,000 or more. After detecting these using Z-scores or IQR, you might decide to:

- **Cap them at the 95th percentile,**
- **Transform them using a log scale if they represent seasonal spikes,**
- **Remove them if they represent data errors or irrelevant noise.**

Outliers are both a challenge and a potential source of insights in machine learning. Properly identifying and handling them is crucial to building robust models that generalize well. Each strategy for handling outliers should align with the specific dataset, goals, and the impact that outliers have on the chosen model.

Methods for Detecting and Removing Outliers

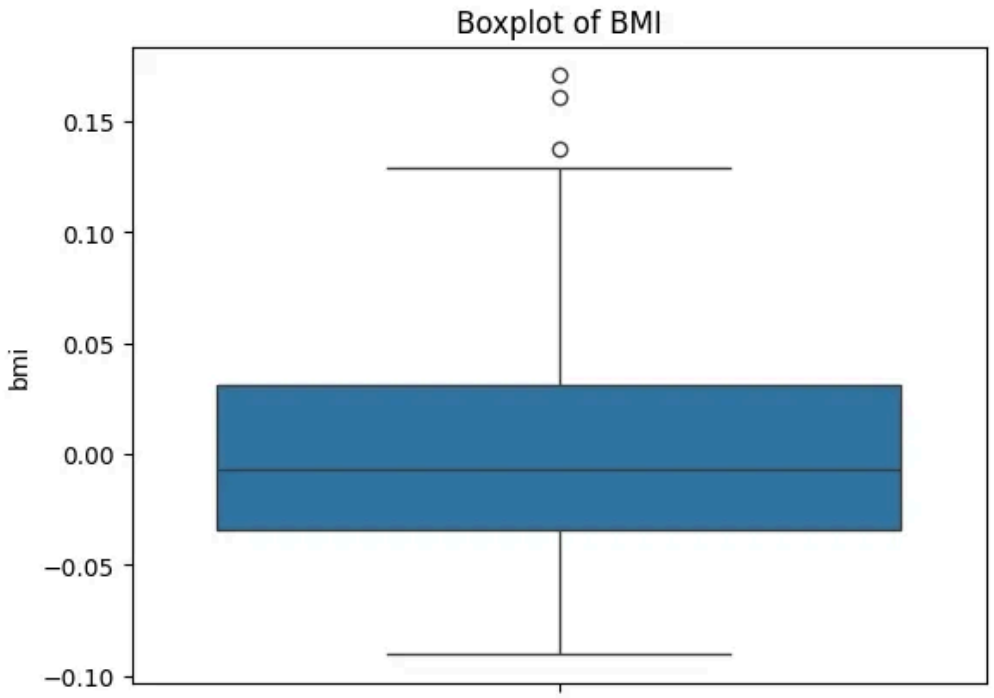
There are several ways to detect and handle outliers in Python. We can use visualization techniques or statistical methods depending on the nature of our data Each method serves different purposes and is suited for specific types of data. Here we will be using Pandas and Matplotlib libraries on the Diabetes dataset which is preloaded in the Sckit-learn library.

1. Visualizing and Removing Outliers Using Box Plots

A boxplot is an effective way for visualizing the distribution of data using quartiles and the points outside the "whiskers" of the plot are considered outliers. They provide a quick way to see where the data is concentrated and where potential outliers lie.

```
import sklearn

from sklearn.datasets import load_diabetes

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

diabetes = load_diabetes()

column_name = diabetes.feature_names

df_diabetics = pd.DataFrame(diabetes.data, columns=column_name)

sns.boxplot(df_diabetics['bmi'])

plt.title('Boxplot of BMI')

plt.show()
```

Output:



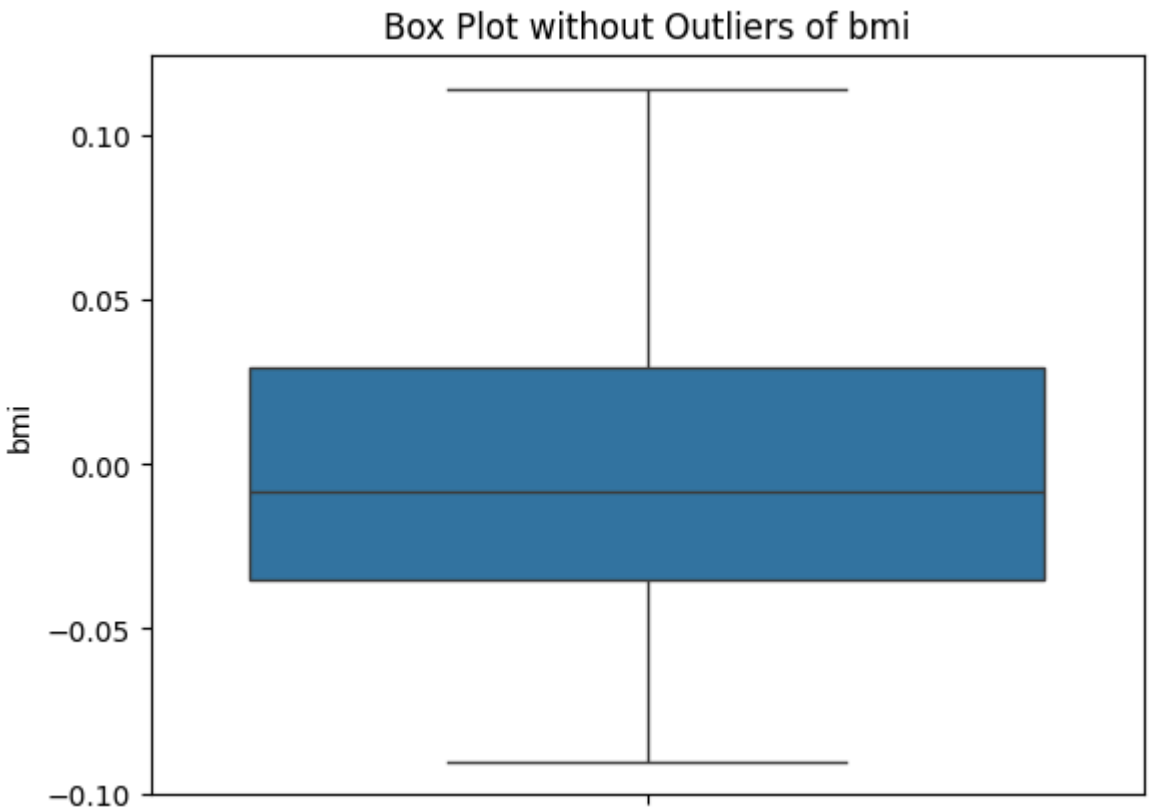
Box Plot

In the boxplot, outliers appear as points outside the whiskers. These values are much higher or lower than the rest of the data. For example, bmi values above 0.12 could be identified as outliers.

To remove outliers, we can define a threshold value and filter the data.

```
def removal_box_plot(df, column, threshold):
    removed_outliers = df[df[column] <= threshold]
    sns.boxplot(removed_outliers[column])
    plt.title(f'Box Plot without Outliers of {column}')
    plt.show()
    return removed_outliers
threshold_value = 0.12
no_outliers = removal_box_plot(df_diabetics, 'bmi', threshold_value)
```

Output:



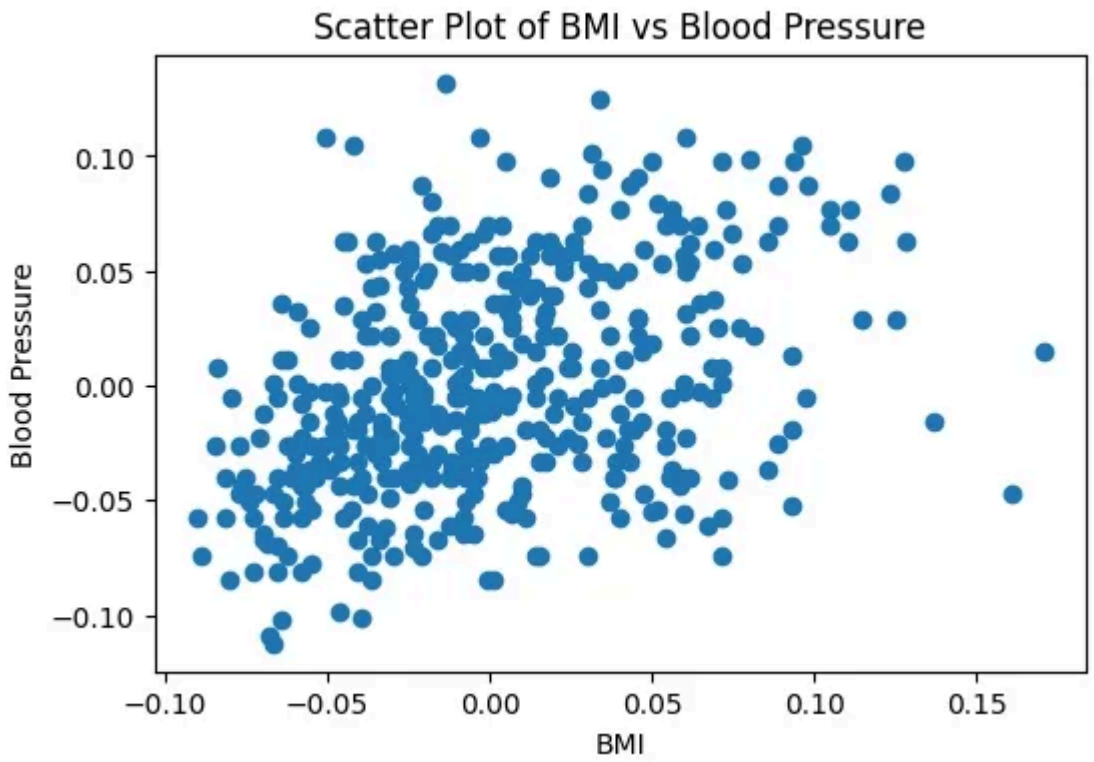
Box Plot

2. Visualizing and Removing Outliers Using Scatter Plots

Scatter plots help visualize relationships between two variables. It is used when we have paired numerical data and when our dependent variable has multiple values for each reading independent variable. Outliers appear as points far from the main cluster of data.

```
fig, ax = plt.subplots(figsize=(6, 4))

ax.scatter(df_diabetics['bmi'], df_diabetics['bp'])

ax.set_xlabel('BMI')

ax.set_ylabel('Blood Pressure')

plt.title('Scatter Plot of BMI vs Blood Pressure')

plt.show()
```

Output:



Looking at the graph can summarize that most of the data points are in the bottom left corner of the graph but there are few points that are exactly opposite that is the top right corner of the graph. Those points in the top right corner can be regarded as Outliers.

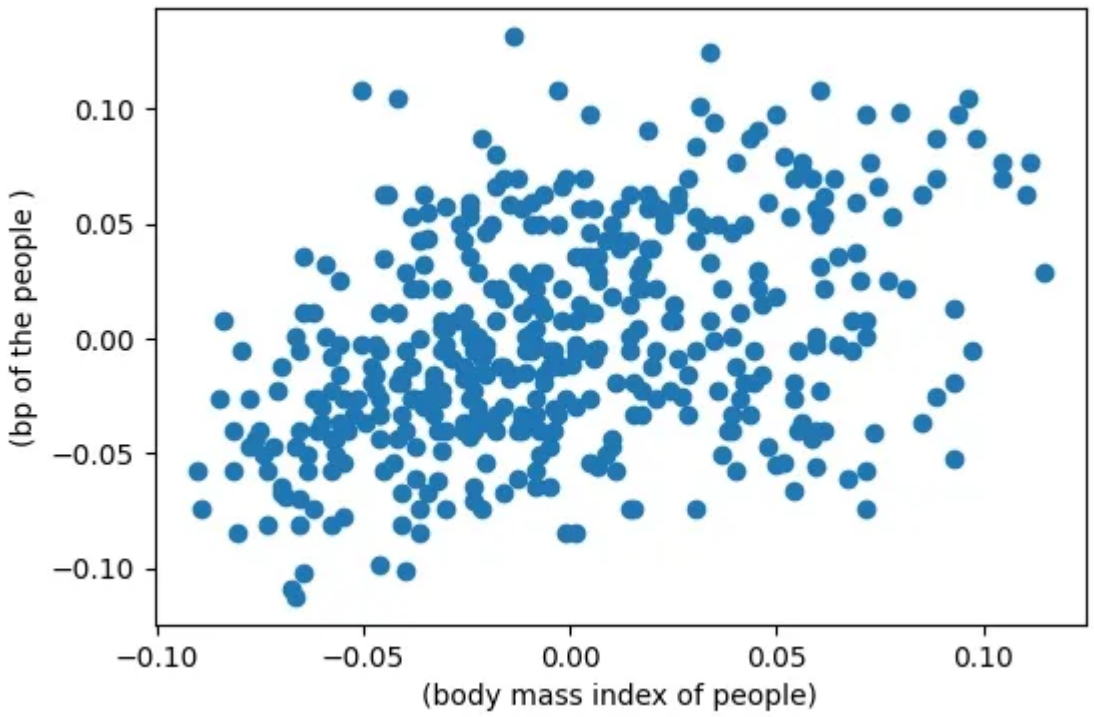
Here's how we can remove the outliers identified visually from the scatter plot.

- **np.where(): Used to find the positions (indices) where the condition is true in the DataFrame.**
- **(df_diabetics['bmi'] > 0.12) & (df_diabetics['bp'] < 0.8): Checks for outliers where 'bmi' is greater than 0.12 and 'bp' is less than 0.8.**

import numpy as np

```
import seaborn as sns
import matplotlib.pyplot as plt
outlier_indices = np.where((df_diabetics['bmi'] > 0.12) & (df_diabetics['bp'] < 0.8))
no_outliers = df_diabetics.drop(outlier_indices[0])
fig, ax_no_outliers = plt.subplots(figsize=(6, 4))
ax_no_outliers.scatter(no_outliers['bmi'], no_outliers['bp'])
ax_no_outliers.set_xlabel('(body mass index of people)')
ax_no_outliers.set_ylabel('(bp of the people )')
plt.show()
```

Output:



Removing Outliers Using Scatter Plots

This removes rows where BMI > 0.12 and BP < 0.8 conditions derived from visual inspection.

3. Z-Score Method for Outlier Detection

Z- Score is also called a standard score. This score measures how far a data point is from the mean, in terms of standard deviations. If the Z-score exceeds a given threshold (commonly 3) the data point is considered an outlier.

$$Z-score=x-\mu\sigma$$

Where:

- **xx = data point**
- **μμ = mean**
- **σσ = standard deviation**

Here we are calculating the Z scores for the 'age' column in the DataFrame df_diabetics using the z-score function from the SciPy stats module. The resulting array z contains the absolute Z scores for each data point in the 'age' column which shows how many standard deviations each value is from the mean.

```
from scipy import stats

import numpy as np

z = np.abs(stats.zscore(df_diabetics['age']))

print(z)
```

Output:

```
0.6477606  0.03680262 0.57415536 1.79607132 0.26591186 1.1087436
0.49778562 1.56696208 0.11593688 1.26148309 0.42141587 0.80050009
0.34228161 0.87686984 0.41865135 1.03237385 0.03680262 0.72413034
0.9560041  0.19230663 0.34504612 0.19230663 0.41865135 1.10597908
0.57415536 1.56696208 2.25428981 0.95323959 0.03956713 0.41865135
0.34228161 0.03956713 0.34228161 1.49059233 1.02960933 0.11317236
0.72413034 0.4950211  0.41865135 0.9560041  1.10597908 0.11593688
0.18954211 0.49778562 0.87963435 1.56696208 0.72413034 1.26148309
1.79607132 1.10597908 0.26591186 1.25871858 0.49778562 0.34228161
2.32789504 0.42141587 0.34504612 1.02960933 1.18511334 0.57139085
1.33508832 1.1087436  0.19230663 0.11317236 1.56419757 1.1087436
1.71693706 0.11593688 0.57415536 1.1087436  0.18954211 0.42141587
0.4950211  0.80050009 1.64333183 0.18954211 0.03680262 1.64333183
0.6477606  0.72689486 1.02960933 0.87963435 0.19230663 1.48782782
0.18954211 0.57415536 0.34228161 0.26867637 1.18511334 0.87686984
0.11593688 0.87686984 0.9560041  0.9560041 ]
```

Z scores

Now to define an outlier threshold value is chosen which is generally 3.0. As 99.7% of the data points lie between ±3±3 standard deviation using Gaussian Distribution approach.

Let's remove rows where Z value is greater than 2.

- **np.where() : Used to find the positions (indices) in the Z-score array where the condition is true.**
- **z > threshold : Checks for outliers in the 'age' column where the absolute Z-score exceeds the defined threshold (typically 2 or 3).**
- **threshold = 2 : A cutoff value used to identify outliers, data points with a Z-score greater than 2 are considered outliers.**

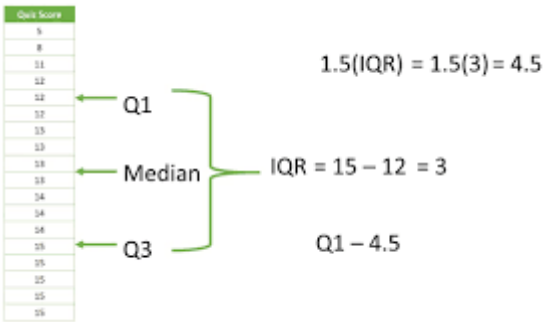
```
import numpy as np

threshold_z = 2

outlier_indices = np.where(z > threshold_z)[0]

no_outliers = df_diabetics.drop(outlier_indices)

print("Original DataFrame Shape:", df_diabetics.shape)
```

print("DataFrame Shape after Removing Outliers:", no_outliers.shape)

Output:

Original DataFrame Shape: (442, 10)
DataFrame Shape after Removing Outliers: (426, 10)

4. Interquartile Range (IQR) Method



IQR (Inter Quartile Range) method is a widely used and reliable technique for detecting outliers.

It is robust to skewed data and helps identify extreme values based on quartiles and it is the most trusted approach used in the research field.

The Interquartile Range (IQR) method provides a robust way to identify and potentially remove outliers from a dataset, especially when the data is not normally distributed or contains skewness.

Steps for applying the IQR method to remove outliers:

Calculate the First Quartile (Q1): This is the 25th percentile of the dataset, meaning 25% of the data falls below this value.

Calculate the Third Quartile (Q3): This is the 75th percentile of the dataset, meaning 75% of the data falls below this value.

Calculate the Interquartile Range (IQR): The IQR is the difference between the third and first quartiles:

Code

```
IQR = Q3 - Q1
```

Determine the Outlier Boundaries:

Lower Bound: This is calculated as Q1 minus 1.5 times the IQR:

Code

```
Lower Bound = Q1 - (1.5 * IQR)
```

Upper Bound: This is calculated as Q3 plus 1.5 times the IQR:

Code

```
Upper Bound = Q3 + (1.5 * IQR)
```

Identify and Handle Outliers: Any data point that falls below the Lower Bound or above the Upper Bound is considered an outlier. Once identified, these outliers can be handled in various ways, such as:

Removal: Deleting the outlier data points from the dataset.

Capping/Winsorizing: Replacing the outlier values with the respective upper or lower bound values.

Transformation: Applying mathematical transformations to the data to reduce the impact of outliers.

SIMPLE EXAMPLE

Detecting Outlier with IQR

IQR is used to measure variability by dividing a data set into quartiles. The data is sorted in ascending order and then we split it into 4 equal parts. The values Q1 (25th percentile), Q2 (50th percentile or median) and Q3 (75th percentile) separate dataset in 4 equal parts.

If a dataset has *2n or 2n+1* data points, then

- Q2 = median of the dataset.
- Q1 = median of n smallest data points.
- Q3 = median of n highest data points.

The IQR is calculated as: IQR=Q3−Q1

Data points that fall below Q1−1.5×IQR or above Q3+1.5×IQR are considered outliers.

Example:

Assume the below data:

6, 2, 1, 5, 4, 3, 50.

If these values represent the number of chapatis eaten in lunch then 50 is clearly an outlier. Let's use Python to detect it.

Step 1: Import necessary libraries.

```
import numpy as np
import seaborn as sns
```

Step 2: Sorting data in ascending order.

```
data = [6, 2, 3, 4, 5, 1, 50]
sort_data = np.sort(data)
sort_data
```

Output:

array([1, 2, 3, 4, 5, 6, 50])

Step 3: Calculating Q1, Q2, Q3 and IQR.

```
Q1 = np.percentile(data, 25, interpolation = 'midpoint')
Q2 = np.percentile(data, 50, interpolation = 'midpoint')
Q3 = np.percentile(data, 75, interpolation = 'midpoint')

print('Q1 25 percentile of the given data is, ', Q1)
print('Q1 50 percentile of the given data is, ', Q2)
print('Q1 75 percentile of the given data is, ', Q3)

IQR = Q3 - Q1
print('Interquartile range is', IQR)
```

Output:

Q1 25 percentile of the given data is, 2.5

Q1 50 percentile of the given data is, 4.0

Q1 75 percentile of the given data is, 5.5

Interquartile range is 3.0

Step 4: Find the lower and upper limits.

```
low_lim = Q1 - 1.5 * IQR
up_lim = Q3 + 1.5 * IQR
print('low_limit is', low_lim)
print('up_limit is', up_lim)
```

Output:

low_limit is -2.0

up_limit is 10.0

Step 5: Identify the outliers.

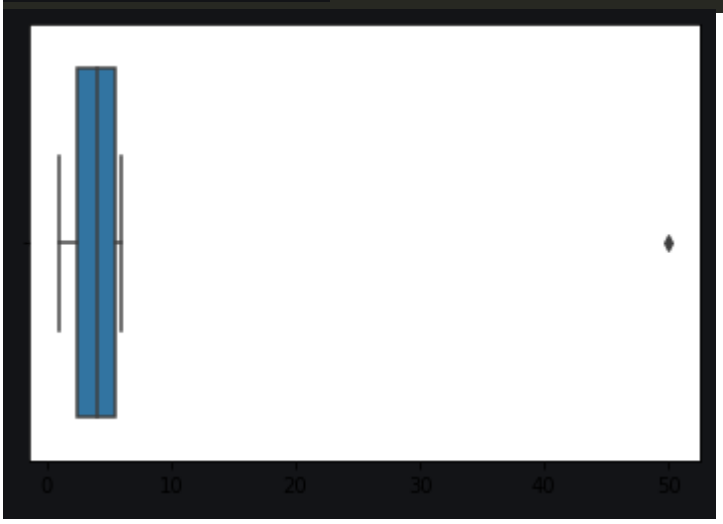
```
outlier =[]
for x in data:
    if ((x> up_lim) or (x<low_lim)):
        outlier.append(x)
print(' outlier in the dataset is', outlier)
```

Output:

outlier in the dataset is [50]

Step 6: Plot the box plot to highlight outliers.

```
sns.boxplot(data)
```



This method along with visualizing data through box plots ensures more reliable and robust data preprocessing.

ANOTHER EXAMPLE

The IQR is calculated as the difference between the third quartile (Q3) and the first quartile (Q1):

IQR=Q3−Q1

Syntax : numpy.percentile(arr, n, axis=None, out=None)

Parameters:

- **arr: Input array.**
- **n: Percentile value.**

Here we are calculating the interquartile range (IQR) for the 'bmi' column in the DataFrame df_diabetics. It first finds the first quartile (Q1) and third quartile (Q3) using the midpoint method then calculates the IQR as the difference between Q3

and Q1 which providing a measure of the spread of the middle 50% of the data in the 'bmi' column.

```
Q1 = np.percentile(df_diabetics['bmi'], 25, method='midpoint')

Q3 = np.percentile(df_diabetics['bmi'], 75, method='midpoint')

IQR = Q3 - Q1

print(IQR)
```

Output:

0.06520763046978838

To define the outlier base value is defined above and below dataset's normal range namely Upper and Lower bounds define the upper and the lower bound (1.5*IQR value is considered i.e:

- **upper=Q3+1.5∗IQR**
- **lower=Q1−1.5∗IQR**

In the above formula the 0.5 scale-up of IQR (new_IQR = IQR + 0.5*IQR) is taken to consider all the data between 2.7 standard deviations in the Gaussian Distribution.

```
upper = Q3+1.5*IQR

upper_array = np.array(df_diabetics['bmi'] >= upper)

print("Upper Bound:", upper)

print(upper_array.sum())


lower = Q1-1.5*IQR

lower_array = np.array(df_diabetics['bmi'] <= lower)

print("Lower Bound:", lower)

print(lower_array.sum())
```

Output:

```
Upper Bound: 0.12879000811776306
3
Lower Bound: -0.13204051376139045
0
```

Upper and Lower bounds

Now lets detect and remove outlier using the interquartile range (IQR).

Here we are using the interquartile range (IQR) method to detect and remove outliers in the 'bmi' column of the diabetes dataset. It calculates the upper and lower limits based on the IQR it identifies outlier indices using Boolean arrays and then removes the corresponding rows from the DataFrame which results in a new DataFrame with outliers excluded. The before and after shapes of the DataFrame are printed for comparison.

```
import sklearn

from sklearn.datasets import load_diabetes

import pandas as pd

diabetes = load_diabetes()


column_name = diabetes.feature_names

df_diabetes = pd.DataFrame(diabetes.data)

df_diabetes .columns = column_name

df_diabetes .head()

print("Old Shape: ", df_diabetes.shape)
```

```
Q1 = df_diabetes['bmi'].quantile(0.25)
Q3 = df_diabetes['bmi'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

upper_array = np.where(df_diabetes['bmi'] >= upper)[0]
lower_array = np.where(df_diabetes['bmi'] <= lower)[0]

df_diabetes.drop(index=upper_array, inplace=True)
df_diabetes.drop(index=lower_array, inplace=True)

print("New Shape: ", df_diabetes.shape)
```

Output:

Old Shape: (442, 10)
New Shape: (439, 10)

With outlier detection and removal we ensure that our data is clean, reliable and ready to provide valuable insights and setting the foundation for robust analysis and accurate models.